

Enhancement Implementation Plan

Gcode Workspaces (gcode-pm-tool) — EN_001 to EN_021

Document status	Draft for approval. No application code or database has been changed.
Prepared	2026-10-01
Scope	Verification of all 21 enhancements, root causes, technical plan for the High-priority items, multi-organization switching, multi-scope role assignments, data-safety and rollout plan.
Evidence base	Source-code audit plus a live Playwright run against the staging database (Next.js 14, Prisma, Postgres on Neon). Screenshots and request logs are kept in the session scratchpad.
What we need from you	Approve / change the decisions on the last pages (Section 11), then implementation starts with Phase 0.

Headline. Of the 21 enhancements, 3 already pass (EN_001, EN_006, EN_007), 2 are partly done and 16 fail. Most High-priority failures share one root cause: the permission model gives each user a single flat role list and a single scope. Fixing that foundation once (multi-scope role assignments) unblocks EN_013, 016, 018, 019, 020 and 021. Multi-organization support is a second, separate foundation. All database changes are designed to be additive so existing production data is never deleted or orphaned.

Contents

1	Executive summary	Results at a glance, what changes, effort and risk
2	Verification results	All 21 items: verdict and live evidence
3	Root-cause findings	Why the High-priority items fail today
4	Target architecture	Multi-scope role assignments and multi-organization accounts
5	Plan per High-priority enhancement	Files, changes, acceptance checks for 12 items
6	Medium / Low items	Recommendations for EN_002, 004, 007, 008, 010, 011, 014
7	Frontend changes	Account-badge org switcher, access editor, permissions context
8	Data safety and migration plan	How production data stays intact
9	Phasing and effort	Order of work, estimates, dependencies
10	Risks and mitigations	What could go wrong and how it is contained
11	Decisions required	Approve or change each default
12	Test plan	Playwright checks that prove each fix
A	Appendix: file inventory	Every file touched, grouped by area

1. Executive summary

Metric	Value
Enhancements reviewed	21 (12 High, 4 Medium, 5 Low)
Passing today	3 (EN_001, EN_006, EN_007)
Partly done	2 (EN_010 native confirm only, EN_015 admin row actions only)
Failing / missing	16
High-priority items	12 (EN_003, 005, 009, 012, 013, 015, 016, 017, 018, 019, 020, 021)
Medium / Low items	9 (EN_001, 002, 011, 014 Medium; EN_004, 006, 007, 008, 010 Low)
Two foundations required	Multi-scope role assignments; multi-organization accounts
Database approach	Additive migrations only; destructive cleanup is a separate, approved release
Estimated effort	About 22 to 30 working days for one developer (Section 9), excluding review/QA cycles

What this plan delivers

- **Correct, granular access.** A person can hold different roles on different scopes (e.g. Program Manager on Workspace A, Team Member on Workspace B).
- **Multiple organizations per person.** A switcher inside the account badge lets someone who belongs to several organizations choose which one they are working in.
- **Server and UI agree.** Buttons a role cannot use are hidden in the UI; the server remains the real gate.
- **Clear feedback.** Uniform success and error toasts for every create, edit and delete.
- **Safer sign-in.** Sessions end when access changes; credentials are protected in transit (Section 5, EN_009).
- **Profile page.** View and edit profile, change password, for admins and users.

What this plan deliberately does not do

- It does not drop or rewrite existing user, role, scope or workspace data in the first release.
- It does not copy the staging database to production. Only migration files travel.
- It does not change email-change flows (needs OTP) - proposed as a follow-up.

2. Verification results

Method: source-code audit, then a live Playwright run on the staging app (localhost:3000) with an Admin, a Portfolio Manager, a Program Manager and a Team Member created through the app. 'Fail' means the expected behaviour was not observed. 'Judgement' means a visual call.

ID	Pri	Enhancement	Result	Live evidence
EN_001	Med	Organization at signup	Pass	Register form has 'Organization Name'; org created and user landed in it.
EN_002	Med	Char limits + list view limit	Fail	No maxlength: 108-char name and 600-char description accepted. List-view long-name case not exercised.
EN_003	High	Breadcrumb truncation + hover	Fail	Full 108-char name rendered in breadcrumb; no ellipsis, no title attribute.
EN_004	Low	Disable Save when unchanged	Fail	Edit Workspace modal: Save enabled with nothing changed.
EN_005	High	Toast on edit/delete	Fail	No toast after unchanged save or rename; delete toast empty.
EN_006	Low	Logo auto-adjust	Pass	1200x100 logo renders 150x30 (fixed height, object-contain).
EN_007	Low	Edit card spacing	Pass (judgement)	Edit modal layout is tidy; visual call only.
EN_008	Low	Live data updates	Fail	Workspace created in a second session did not appear in open tab after 8 s.
EN_009	High	Credential encryption	Fail	Login body is plain JSON {email, password}; register body includes password in clear.
EN_010	Low	Confirm modal on deactivate/delete	Partial	Uses browser confirm() dialogs; no custom modal.
EN_011	Med	Role filter shows all roles	Fail	After selecting Program Manager the list collapses to All Roles + Program Manager; Admin disappears.
EN_012	High	Remove Custom Roles section	Fail	Role Definitions tab still shows Custom Roles and '+ Create Custom Role'.
EN_013	High	No scope for Admin	Fail	POST /api/team/invite-codes with roles [Admin] + workspace scope returned 201.
EN_014	Med	Actions UI in Team Mgmt	Fail	At 1000 px the Actions column is clipped off the right edge.
EN_015	High	Profile view/edit/reset	Partial	Only admin row actions exist; /profile, /account, /settings, /me all 404; menu has only 'Sign out'.
EN_016	High	Frontend role-based visibility	Fail	Program Manager sees 'New Workspace' button and card menu.
EN_017	High	App-wide CRUD toasts	Fail	Create workspace shows no toast; toast layer only reports errors.
EN_018	High	Auto sign-out on role change	Fail	After role change the user's existing session stayed signed in.
EN_019	High	Portfolio Manager org-wide access	Fail	Workspace-scoped Portfolio Manager created a workspace (201) but it is absent from their own list.
EN_020	High	Program/Initiative permissions	Fail	Permissions matrix has no Program or Initiative rows.
EN_021	High	Person responsible gets task CRUD	Fail	Assigned Team Member got 403 on PATCH (edit), POST and DELETE of the task.

3. Root-cause findings

3.1 One flat role list and one scope per user

`prisma/schema.prisma`, `model User: roles String[]` and `scope String?`. Roles are one flat list; the single scope applies to all of them. It is impossible to express 'Program Manager on A, Team Member on B'. 111 references to `.scope`, `.roles` and the scope helpers exist across 40 files, which is why this must be fixed once, centrally.

3.2 No multi-organization choice

A User row is unique per (`organizationId`, `email`), so the same email can already exist in several organizations, but `api/auth/login/route.ts` signs into the first row whose password matches. With the same password in both organizations the user always lands in the same one and cannot choose. The schema comment itself notes 'no org switcher'.

3.3 Why EN_019 happens

`POST /api/workspaces` creates a workspace but gives the creator no access to it. A workspace-scoped Portfolio Manager creates a workspace outside their own scope; the scope-pruned tree then hides it, which surfaces as 403/missing on view.

3.4 Why EN_021 happens

`api/tasks/[id]/route.ts` authorises purely by capability. The 'Execute Assigned Tasks' capability only allows changing `status` (explicitly noted in code), and nothing checks `task.responsibleId === user.id`.

3.5 Why EN_016 happens

The only UI gating today is the sidebar 'Team Management' link and the `/team` page redirect. Create/edit/delete controls render for every role and rely on the API to return 403.

3.6 Why EN_005 / EN_017 happen

`ActionToast.tsx` exposes `reportIfActionFailed` and `showActionError`; only `DirectoryTab` ever calls the notice variant. There is no success tone and the create/edit/delete paths never trigger one.

3.7 Why EN_018 happens

The 'edit' action in `api/team/users/[id]/route.ts` updates roles and scope only. It never sets `revokedAt` on the user's sessions.

4. Target architecture

4.1 Multi-scope role assignments (Foundation A)

	Today	Proposed
Storage	User.roles String[] + User.scope String?	New table RoleAssignment(id, userId, role, scope?) - one row per role-on-scope
Example	Program Manager + Team Member, both on workspace A	Program Manager @ Workspace A; Team Member @ Workspace B; (optional) Portfolio Manager @ Whole Org
Session user	{ roles[], scope }	{ assignments[{role, scope}] }
Permission check	capabilities of union(roles), then scope check on one scope	canOnNode(user, capability, node): any assignment whose role has the capability AND whose scope covers the node
Visibility	Tree pruned to one scope chain	Tree pruned to the union of scope chains from all assignments
Assignee picker	User.scope covers node	Any of the candidate's assignments covers node
Rules enforced	Only 'Client needs a scope'	Admin and Portfolio Manager: scope must be null (whole org). No duplicate role+scope. At least one assignment.

Semantics: permissions on a node are the union of every assignment that covers it. There is no 'active scope' switch - a person with Program Manager on A and Team Member on B simply sees both subtrees with the right controls on each. Ancestors of a scope stay view-only (existing behaviour preserved).

4.2 Multi-organization accounts (Foundation B)

	Today	Proposed
Identity	One User row per org per email, each with its own passwordHash	Account(id, email unique, passwordHash) + User becomes the per-org membership with accountId
Login	Checks password against every row with that email; first match wins	Authenticates the Account; lists active memberships
One membership	Signs in directly	Signs in directly (unchanged experience)
Several memberships	Not selectable	Opens last-used org (cookie gcode_last_org) or /select-org picker
Switching later	Not possible without signing out and back in	Org switcher inside the account badge dropdown; POST /api/auth/switch-org
Invite existing person	Creates a new user with a new password	Detects existing Account, adds a membership; invite page shows 'Join Org as you@x.com' (no new password)
Register org with known email	Creates independent account	Attaches new org to the existing Account after password confirmation
Session	Session -> User (membership)	Unchanged: Session -> User. Switching revokes the old session and creates one for the chosen membership.

Tenant isolation is unchanged. Every API still filters by the session user's organizationId. Switching organization creates a brand-new session for that membership; roles and scopes from one organization can never leak into another.

5. Plan per High-priority enhancement

Each item lists the intended behaviour, the concrete changes, and the acceptance check that will prove it. File paths are relative to `src/`.

EN_003 — Breadcrumb truncation with hover

Behaviour	Each crumb is capped (about 24 characters, CSS truncate with ellipsis); hovering shows the full name. e.g. 'My Name is Lavya, I am an BE graduate from CU' shows 'My Name is Lavya, I am...'.
Changes	Add max-width + truncate to each crumb and to the last crumb; add title attribute with the full label; keyboard focus shows the same tooltip.
Files	components/Breadcrumbs.tsx
Acceptance	On a 108-char workspace name the crumb is under 24 chars visible, title equals the full name, no horizontal overflow at 1000 px.

EN_005 / EN_017 — Success and error toasts for every create / edit / delete

Behaviour	Every successful create, edit, delete and role/team change shows a short success toast; failures show the error toast (already exists). Toasts stack, auto-dismiss, and are announced to screen readers.
Changes	Add <code>showActionSuccess</code> + success tone to the toast layer; add a <code>mutate(res, {success})</code> helper beside <code>reportIfActionFailed</code> ; wire it into every write call; use <code>aria-live</code> ; allow more than one toast at a time.
Files	components/ActionToast.tsx, CreateItemModal.tsx, HierarchyLevelClient.tsx, HierarchyCardGrid.tsx, TaskDetailPanel.tsx, TaskKanbanBoard.tsx, team/{TeamManagementClient,DirectoryTab,EditUserModal,InviteUserModal,InviteCodesTab,RoleDefinitionsTab}.tsx
Acceptance	Creating, renaming and deleting a workspace/initiative/program/project/task each show a toast; team invite, edit, deactivate, delete, revoke-code each show a toast; failures still show the error toast.

EN_009 — Protect login credentials in transit

Behaviour	Passwords are not readable in the request payload shown by browser tools or captured by intermediaries.
Changes	Recommended baseline: enforce HTTPS + HSTS, ensure request bodies are never logged. If app-layer encryption is required: fetch a server public key, encrypt {email,password,nonce,timestamp} with WebCrypto RSA-OAEP, server decrypts and rejects replays (nonce + 60 s window). Note: this hides the password from network intermediaries but not from the user's own dev tools, and does not replace TLS.
Files	(auth)/login/page.tsx, register/page.tsx, join/page.tsx, invite/[token]/page.tsx, api/auth/{login,register-org,complete-invite,join-with-code}/route.ts, new lib/auth/payloadCrypto.ts
Acceptance	Login/register/join/invite-complete requests carry an opaque ciphertext field; old plain-JSON requests are rejected after a transition period; replayed request is rejected.

EN_012 — Remove Custom Roles section

Behaviour	Team Management no longer shows Custom Roles; built-in role matrix remains.
Changes	Remove the Custom Roles block and create-button from the Role Definitions tab; stop exposing the create API; keep the CustomRole table and keep resolving capabilities for anyone who already holds a custom role (no data deleted).
Files	components/team/RoleDefinitionsTab.tsx, team/TeamManagementClient.tsx, api/team/custom-roles/route.ts, lib/auth/capabilities.ts (read-only compatibility)
Acceptance	Role Definitions tab shows only the matrix; GET/POST custom-roles endpoint no longer reachable from UI; existing custom-role holders keep their permissions.

EN_013 — Admin cannot be assigned a workspace scope

Behaviour	An Admin assignment is always whole-org. Workspace-specific Admin cannot be created through invite, invite code, or edit.
Changes	Server validation: reject (400) any assignment where role = Admin and scope is not null, in invite, invite-codes and user edit routes. UI: when Admin is picked, the scope control locks to 'Whole Org'. Existing scoped Admin rows are widened to whole-org with the old value logged (decision D2).
Files	api/team/{invite,invite-codes,users/{id}}/route.ts, team/{InviteUserModal,InviteCodesTab,EditUserModal}.tsx, new components/team/RoleAssignmentsEditor.tsx
Acceptance	API returns 400 for Admin+scope on all three routes; UI locks scope; audit table lists any changed rows.

EN_015 — Profile page: view, edit, reset password

Behaviour	Every signed-in user (admins included) can open a Profile page, edit their name, and change their password by giving the current one.
Changes	New /profile page; GET/PATCH /api/me (name); POST /api/me/password (current+new, min length, hash, revoke other sessions); 'Profile' item in the account menu. Email change is deferred (needs OTP).
Files	new (app)/profile/page.tsx, ProfileClient.tsx, api/me/route.ts, api/me/password/route.ts; components/UserMenu.tsx; lib/auth/password.ts (reuse)
Acceptance	/profile renders for all roles; wrong current password is rejected; success revokes other sessions and shows a toast.

EN_016 — Role-based visibility in the frontend

Behaviour	Controls a role cannot use are hidden (e.g. 'New Workspace' for Program Manager). The server still enforces everything.
Changes	Add a PermissionsProvider with useCan(capability, node?). The server stamps each tree node with perms {edit, delete, createChild} computed from the user's assignments. Gate create buttons, card/row menus, task edit/delete and sidebar entries.
Files	new components/PermissionsContext.tsx, (app)/layout.tsx, lib/tree.ts, HierarchyLevelClient, HierarchyCardGrid, HierarchyTreeTable, TaskDetailPanel, TaskKanbanBoard, Sidebar
Acceptance	Program Manager (workspace-scoped) sees no 'New Workspace' and no workspace edit/delete menu; sees 'New Project' inside their scope; Team Member sees no create buttons.

EN_018 — Auto sign-out on role update

Behaviour	When an admin changes someone's role or scope, that person is signed out and sees a clear message the next time they act.
Changes	Revoke all of the user's sessions inside the 'edit' action. The (app) layout redirects to /login?reason=access-changed when the session is missing; a fetch wrapper turns any 401 into the same redirect plus a toast.
Files	api/team/users/{id}/route.ts, (app)/layout.tsx, login/page.tsx, new lib/client/apiFetch.ts
Acceptance	After an admin edit, the target user's next navigation or API call lands on /login with the explanatory message; the admin's own session is unaffected.

EN_019 — Portfolio Manager has whole-organization access

Behaviour	Portfolio Manager is always org-wide; it can create and see every workspace, including ones it just created.
Changes	Enforce scope = null for Portfolio Manager (same validators as EN_013); hide the scope control for it; migrate existing scoped Portfolio Managers per decision D2 with the old scope logged.
Files	api/team/**, api/workspaces/route.ts, lib/auth/scope.ts, components/team/RoleAssignmentsEditor.tsx
Acceptance	A Portfolio Manager creates a workspace and immediately sees and opens it; no scoped Portfolio Manager can be created.

EN_020 — Program and Initiative CRUD permissions

Behaviour	Programs and Initiatives get their own explicit capabilities in the permission matrix.
Changes	Add capabilities 'Create/Edit Initiatives' and 'Create/Edit Programs' (optionally Delete, decision D6). Seed RoleCapability rows with sensible defaults (Portfolio Manager: both; Program Manager: Programs). Initiative/program routes use the new capabilities instead of the workspace/project ones. The matrix UI reads the constants, so rows appear automatically.
Files	lib/auth/capabilityConstants.ts, prisma/seed-rbac.ts, api/initiatives/**, api/programs/**, team/RoleDefinitionsTab.tsx
Acceptance	Matrix shows the new rows; toggling them changes what each role can do on the initiative/program routes.

EN_021 — Person responsible gets CRUD on that task

Behaviour	Whoever is set as 'Person responsible' can view, edit and delete that specific task regardless of their role, and only that task.
Changes	In tasks/[id] PATCH and DELETE: allow when task.responsibleId equals the caller; scope check is bypassed only for that task. UI mirrors it via perms. Creating new tasks is not implied (the task already exists).
Files	api/tasks/[id]/route.ts, TaskDetailPanel.tsx, TaskKanbanBoard.tsx, TaskListView.tsx, lib/tree.ts
Acceptance	A Team Member assigned to a task can edit every field and delete it; the same user gets 403 on a task assigned to someone else.

6. Medium and Low items

ID	Pri	Recommendation	Files
EN_001	Med	Already works. Add: inviting an existing email attaches a membership (Foundation B).	see Section 4.2
EN_002	Med	Max 60 chars for names/titles, 255 for descriptions: maxlength on inputs, counters, and matching server validation (400). List and card views clamp text with ellipsis and a title tooltip.	CreateItemModal.tsx, api/*/route.ts, HierarchyCardGrid.tsx, HierarchyTreeTable.tsx
EN_004	Low	Track initial values; Save disabled until a field differs. Apply to every edit modal.	CreateItemModal.tsx, team/EditUserModal.tsx
EN_007	Low	Passes visually; optional polish of padding and description textarea height.	CreateItemModal.tsx
EN_008	Low	Cheapest: refetch on window focus and a 30 s poll on list/tree views. Later: SSE for instant updates.	lib/tree.ts, HierarchyLevelClient.tsx, new hook useLiveRefresh
EN_010	Low	Replace native confirm() with the existing ConfirmDeleteModal component for deactivate, delete and revoke.	team/DirectoryTab.tsx, team/InviteCodesTab.tsx, ConfirmDeleteModal.tsx
EN_011	Med	Role filter lists all built-in roles always (static list), matching users with any assignment of that role.	team/DirectoryTab.tsx, team/TeamManagementClient.tsx
EN_014	Med	Wrap or truncate chips, allow horizontal scroll or sticky Actions column, make the access column compact (Section 7).	team/DirectoryTab.tsx

7. Frontend changes

7.1 Org switcher in the account badge

Element	Behaviour
Header	Org name next to the user name becomes a trigger when the account has 2+ memberships; plain text with one.
Dropdown (avatar)	Profile block (name, email) SWITCH ORGANIZATION list (initials tile, check on active, 'invited' rows disabled) Profile Sign out.
Switch action	POST /api/auth/switch-org {userId}; spinner on the row; on success push /workspaces, refresh, toast 'Switched to Acme Corp'. URL is not preserved (slugs are per org).
Login	If several memberships: open last-used org (cookie) else /select-org page.
States	Single org (no switcher), invited/suspended (disabled with label), failure (error toast), removed membership (redirect to /select-org).
Accessibility	role=menu, arrow-key navigation, Esc to close, scroll after about 6 orgs.

File	Change
(app)/layout.tsx	Load memberships for the account and pass to Header; wrap content in PermissionsProvider keyed by userId.
components/Header.tsx	Accept memberships/currentUserId; org name becomes the switcher trigger; second line shows highest role + '+N access'.
components/UserMenu.tsx	Rebuild as a full menu (profile, orgs, Profile link, Sign out).
components/OrgSwitcher.tsx (new)	Switch call, loading and toast handling.
(auth)/select-org/page.tsx (new)	Full-page picker when no remembered org.
(auth)/login/page.tsx	Handle multi-membership response.
(auth)/invite/[token]/page.tsx	'Join Acme Corp as you@x.com' confirm for existing accounts.

7.2 Multi-scope access editor

Edit QA User

ACCESS

```
+-----+-----+-----+
| Program Manager v | Workspace A v | x |
| Team Member v | Workspace B > Init 1 v | x |
| Portfolio Mgr v | Whole Org (locked) | x |
+-----+-----+-----+
+ Add access
i Saving signs this person out so new access applies.
[Cancel] [Save - disabled until changed]
```

File	Change
components/team/RoleAssignmentsEditor.tsx (new)	Rows of role + scope (reuses ScopeSelect); Admin and Portfolio Manager lock scope to Whole Org; blocks duplicates; needs at least one row.
team/EditUserModal.tsx	Uses the editor; Save disabled until changed (EN_004); shows the sign-out notice (EN_018).
team/InviteUserModal.tsx, InviteCodesTab.tsx	Same editor; invite codes keep 'Client needs a scope'.
team/types.ts	TeamUser gains assignments[{role, scope, scopeLabel}]; roles/scope removed after cutover.
team/DirectoryTab.tsx	Role and Scope columns merge into 'Access': two chips (e.g. 'Program Manager - WS-A') plus '+N' popover; role filter static (EN_011); layout fix (EN_014).
lib/auth/scopeLabel.ts	Batch-resolve labels for all assignments of a user.
components/PermissionsContext.tsx (new)	useCan(capability, node?) reads assignments and server-stamped perms.

File	Change
lib/tree.ts	Prune to union of scope chains; stamp perms on each node.

8. Data safety and migration plan

Hard requirement: existing production data must never be deleted or orphaned. The staging database is never copied over production; only migration files are applied (`prisma migrate deploy`, which already runs in `npm run build`).

8.1 Release shape: expand, then contract

Release	Database	Application	Reversible?
R1 (expand)	Add Account, RoleAssignment, ScopeChangeLog and nullable User.accountId. Backfill by copying. No DROP, no rename, no type change.	Dual-write old and new; reads prefer new with fallback to old columns.	Yes - roll back code, nothing lost
R2 (verify)	None	Monitor: assignments match legacy columns; org switcher live.	Yes
R3 (contract, optional)	Drop User.roles / User.scope only after sign-off.	Reads only the new tables.	Needs restore from backup

8.2 Rules built into the migrations

- **Additive only.** No DROP, TRUNCATE, DELETE or column type change on existing tables in R1.
- **Copy, don't move.** Each existing User gets RoleAssignment rows from its current roles x scope. Idempotent SQL (`INSERT ... WHERE NOT EXISTS`).
- **Account backfill never merges unsafely.** Rows grouped by lower-cased email. Single row -> its own Account (password copied). Same email with different passwords stays as separate Accounts until the person confirms a merge by email. Email-less guest users get no Account and work as today.
- **No dangerous cascades.** Account -> User uses `ON DELETE SET NULL`; deleting an Account cannot delete memberships or workspaces. RoleAssignment -> User cascades (derived data only).
- **Self-checking.** The migration ends with assertions inside the transaction: every user has at least one assignment; no assignment points at a missing user; counts of User, Workspace, Initiative, Program, Project, Task are unchanged. Any failure rolls the whole migration back.
- **Legacy fields untouched.** Workspace.organizationId and Person.organizationId (nullable legacy) are not modified.
- **Forbidden on production:** migrate reset, db push --force-reset, migrate dev, editing or reordering already-applied migrations.

8.3 Data-changing behaviours (need approval)

Item	What changes for existing rows	Safeguard
EN_013	Admins that currently carry a workspace scope become whole-org.	Old value stored in ScopeChangeLog; reversible by script.
EN_019	Scoped Portfolio Managers become whole-org (default) or Program Manager.	Old value stored in ScopeChangeLog.
EN_012	Custom Roles UI hidden.	CustomRole table kept; holders keep permissions; nothing deleted.
EN_018	Users are signed out when access changes.	Sessions revoked, no user data deleted.
Multi-org login	Same-email people see an org picker.	No merging of differing passwords.

8.4 Proof before production

- Rehearse every migration on staging and on a Neon branch copy of production; compare before/after row counts and show you the report.
- Read-only audit script run against production first: row counts, same-email/different-password cases, scoped Admins and Portfolio Managers, dangling scopes.

- Neon backup or branch of production taken immediately before the first production deploy.
- Each migration ships with a documented rollback.

9. Phasing and effort

Estimates are for one developer, in working days, and exclude your review time. They are planning figures, not commitments.

Phase	Scope	Items	Days	Depends on
0 - Safety	Audit script, additive migration (R1), rehearsal on staging, e2e harness from existing Playwright scripts	-	3 - 4	-
1 - Foundation A	RoleAssignment model, canOnNode, capability resolver, session assignments, tree union pruning, dual-write	EN_013, 019, 020 (base)	5 - 7	Phase 0
2 - Quick wins	Toast layer and wiring, breadcrumb truncation, remove Custom Roles, Admin/Portfolio validators, session revoke + 401 handling	EN_003, 005, 017, 012, 013, 018	4 - 5	Phase 1 (for 013, 018)
3 - Access UI	RoleAssignmentsEditor, DirectoryTab Access column, PermissionsProvider + perms, capabilities for Programs/Initiatives, task responsible rule	EN_016, 019, 020, 021, 011, 014, 004	5 - 7	Phase 1
4 - Profile	Profile page, /api/me, password change	EN_015	2	Phase 2 (toasts)
5 - Foundation B	Account model, login flow, switch-org, select-org, invite/register changes, account-badge switcher	Multi-org	4 - 5	Phase 0
6 - Security	HTTPS/HSTS review, optional payload encryption	EN_009	1 - 3	-
7 - Medium/Low	Limits, confirm modal, live refresh polish	EN_002, 008, 010	2 - 3	Phase 2
Total			22 - 30	

10. Risks and mitigations

Risk	Likelihood	Impact	Mitigation
Backfill bug leaves a user without an assignment	Low	High	In-migration assertions + fallback reads from legacy columns + rehearsal on production copy.
111 call sites miss the new permission helper	Medium	High	Single canOnNode helper; grep-based CI check that nothing reads user.scope directly; e2e role matrix.
Migration locks tables on a large dataset	Low	Medium	Additive ALTERs are fast; check production table sizes first; run off-peak.
Same-email accounts with different passwords	Medium	Medium	No automatic merge; email-confirmed merge flow.
Sign-out on access change annoys users	Medium	Low	Clear message and toast; only triggers on actual access change.
App-layer payload encryption gives false confidence (EN_009)	Medium	Medium	Document clearly; TLS remains the primary control.
Frontend hides a control the server still allows (or vice versa)	Medium	Medium	Server stays the source of truth; e2e compares UI visibility with API results per role.
Production deploy runs migration unexpectedly (build runs migrate deploy)	Medium	High	Backup first; deploy R1 on its own; review migration SQL before merging.

11. Decisions required

Please mark each as **Approve default** or write your change. Implementation of the affected phase waits on the answer.

#	Decision	Default (recommended)	Alternatives	Your answer
D1	Multi-org identity model	Shared Account across organizations; conflicting passwords never auto-merged	Per-org password with re-prompt on switch	
D2	Existing scoped Portfolio Managers and Admins	Make whole-org, log the old value	Convert Portfolio Manager to Program Manager; leave as-is and only block new ones	
D3	EN_009 credential protection	HTTPS + HSTS + no body logging	Add WebCrypto RSA-OAEP payload encryption with replay protection	
D4	EN_021 person-responsible rights	Full edit and delete on that one task	Edit only; edit + status only	
D5	Old columns User.roles / User.scope	Keep for at least one release (dual-write), drop later with approval	Drop immediately after backfill	
D6	EN_020 permission granularity	Combined 'Create/Edit' for Initiatives and Programs (matches existing style)	Separate Create / Edit / Delete capabilities	
D7	Login with several organizations	Auto-open last-used org (cookie); picker only if none remembered	Always show the picker	
D8	Invited / suspended orgs in switcher	Show disabled with a label	Hide them	
D9	EN_015 email change	Defer (needs OTP flow)	Include in this scope	
D10	EN_008 live updates	Refetch on focus + 30 s poll now; SSE later	SSE immediately	
D11	EN_002 limits	60 chars name/title, 255 description	50 chars; other numbers	
D12	Before production deploy	Neon backup/branch + read-only audit run by you	Skip audit (not recommended)	

Approval

	Name / date / signature
Approved as written	
Approved with the changes noted above	
Not approved - discuss	

12. Test plan

The Playwright checks used for this verification become a repeatable end-to-end suite. Test users: Admin, Portfolio Manager, Program Manager (scoped), Team Member (scoped and responsible). Run on staging after each phase.

Check	Method	Pass condition
EN_003	Create 108-char name, open detail page	Crumb truncated, title = full name, no overflow at 1000 px
EN_004	Open edit, do not change, inspect Save	Save disabled; enabled after any change
EN_005 / 017	Create, edit, delete at every level; team actions	Success toast for each; error toast still shown on a forced 403
EN_009	Capture login and register request bodies	No readable password; replay rejected
EN_011	Filter Program Manager, reopen list	All built-in roles always present
EN_012	Open Role Definitions	No Custom Roles section
EN_013	POST Admin + scope on invite, invite-code, edit	400 on all; UI locks scope
EN_014	Team table at 1000 and 1366 px	Actions always reachable
EN_015	Open /profile, edit name, change password	Saves; wrong current password rejected; other sessions revoked
EN_016	Compare UI controls to API results per role	No visible control that the API would 403
EN_018	Admin edits user; user acts with old session	Redirect to /login with message
EN_019	Portfolio Manager creates workspace	Visible and openable immediately
EN_020	Open matrix; toggle Programs/Initiatives	Rows exist; routes honour them
EN_021	Responsible user edits and deletes own task; other task	Own: success; other: 403
Multi-scope	User with PM@A and Team Member@B	Sees both subtrees; PM controls only under A
Multi-org	One email in two orgs; switch from badge	Correct org data after switch; no cross-org leakage
Migration	Row counts before/after on a production copy	Identical counts; assertions pass

Appendix A. File inventory

Area	Files (relative to src/ unless noted)
Database	prisma/schema.prisma, new migration (Account, RoleAssignment, ScopeChangeLog), prisma/seed-rbac.ts, new audit script
Auth core	lib/auth/{session,scope,capabilities,capabilityConstants,requireCapability,scopeLabel,password}.ts, new lib/auth/payloadCrypto.ts
Auth routes	api/auth/{login,register-org,register-org/verify-otp,complete-invite,join-with-code,forgot-password}/route.ts, new api/auth/{orgs,switch-org}/route.ts, new api/me/**
Team routes	api/team/{invite,invite-codes,users,users/[id],assignable-users,export,custom-roles}/route.ts
Hierarchy routes	api/{workspaces,initiatives,programs,projects,tasks}/route.ts and their [id]/route.ts
Server pages / layout	(app)/layout.tsx, (app)/workspaces/**/page.tsx, (app)/team/page.tsx, new (app)/profile/**, new (auth)/select-org/page.tsx, (auth)/{login,register,join,invite/[token]}/page.tsx
Data/tree	lib/tree.ts, lib/types.ts
Shared UI	components/{ActionToast,Breadcrumbs,Header,UserMenu,Sidebar,CreateItemModal,ConfirmDeleteModal,PageHeader}.tsx, new components/{PermissionsContext,OrgSwitcher}.tsx, new lib/client/apiFetch.ts
Hierarchy UI	components/{HierarchyLevelClient,HierarchyCardGrid,HierarchyTreeTable,TaskDetailPanel,TaskKanbanBoard,TaskListView}.tsx
Team UI	components/team/{TeamManagementClient,DirectoryTab,EditUserModal,InviteUserModal,InviteCodesTab,RoleDefinitionsTab,ScopeSelect,types}.ts(x), new components/team/RoleAssignmentsEditor.tsx
Tests	new tests/e2e/** (Playwright), seeded role users

Appendix B. Staging test data created during verification

The verification created on the staging database: organization 'QA Playwright Org', one Admin, three role users (Portfolio Manager, Program Manager, Team Member), a few workspaces, and one initiative/program/project/task chain. It is disposable and entirely inside that one organization. No production data was accessed.